BrainStorm / Мозговой Штурм **Realtime AI-powered quiz platform — build a game on any topic and play with friends from any device.** **Платформа интерактивных викторин в реальном времени: играй с друзьями с любого устройства, а вопросы генерирует GigaChat.** [![CI](https://github.com/geff1778/Brainstorm-test/actions/workflows/ci.yml/badge.svg)](https://github.com/geff1778/Brainstorm-test/actions/workflows/ci.yml) [![License: MIT](https://img.shields.io/badge/license-MIT-blue.svg)](LICENSE) ![Python](https://img.shields.io/badge/Python-3.10%2B-3776AB?logo=python&logoColor=white) ![Flask](https://img.shields.io/badge/Flask-3.x-000000?logo=flask) ![Socket.IO](https://img.shields.io/badge/Socket.IO-realtime-010101?logo=socket.io) ![GigaChat](https://img.shields.io/badge/GigaChat-AI-21A038) ![Tests](https://img.shields.io/endpoint?url=https://raw.githubusercontent.com/geff1778/Brainstorm-test/main/.github/badges/tests.json) **Topics:** `flask` · `socketio` · `realtime` · `quiz` · `gigachat` · `python` · `education` · `pwa`

Играть онлайн / Play online

Production: https://brainstorm-c0ap.onrender.com/

Отсканируй QR-код телефоном, чтобы открыть игру · Scan the QR code with your phone to open the game.

Contents / Содержание

- English — What is BrainStorm? · Features · Architecture · Local setup · Environment variables · Tests & quality · Production · API & event reference
- Русский — Что такое BrainStorm? · Возможности · Архитектура · Локальный запуск · Переменные окружения · Тесты и качество · Продакшен
- Roadmap · Contributing · Security · Changelog

English

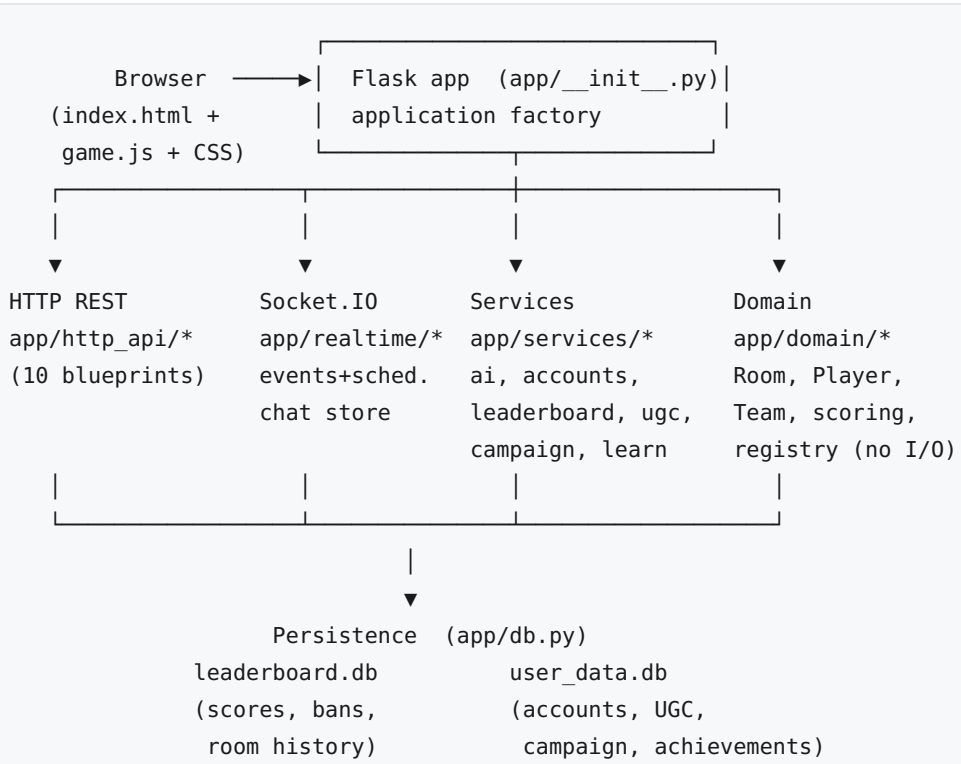
What is BrainStorm?

BrainStorm is a self-hosted, real-time multiplayer quiz. A host creates a room, shares a six-character code, and everyone answers the same questions at the same time from their phone or laptop. Questions are produced by **GigaChat** (Sberbank) for any topic you type, with a built-in offline question bank as a fallback so the game always works — even with no AI key.

Features

Area	Highlights
Gameplay	Six modes — Classic, FFA, Team draft, Lives (elimination), Co-op, and Svoya Igra (Jeopardy-style board). Adaptive difficulty, streaks, time bonuses, bonus questions, jokers, hints, question rephrasing.
AI	GigaChat question generation, hints and rephrasing; Markdown-fenced JSON parsing; automatic fallback bank; per-user rate limiting.
Accounts	Register/login, XP and levels, coins, cosmetics shop, match history, achievements.
Social	In-room chat with moderation, reactions, global leaderboard, room history.
UGC	Players submit their own questions; an admin moderation queue approves or rejects them.
Campaign	12 single-player levels across 3 worlds with stars, boss fights and coin rewards.
Ops	Admin panel (kick/ban/impersonate/live stats), cheat/tester mode, health probes, structured logging, PWA (installable + offline shell).

Architecture



Layering rule: `domain` has no I/O, `services` depend on `domain` + `db`, `http_api`/`realtime` are thin adapters over `services`. `config`, `security`, `errors`, `validation` and `logging_setup` are cross-cutting.

Project layout

```
app/
├── __init__.py          # create_app() factory + SocketIO wiring
├── config.py            # typed settings + production fail-fast validation
├── security.py          # password hashing, rate limiting, escaping
├── validation.py        # boundary input normalisation
├── errors.py            # typed domain errors
├── db.py                # SQLite schema + connection helpers
├── cache.py             # bounded TTL cache
├── logging_setup.py     # structured logging
├── domain/              # pure game logic (no network, no DB)
│   ├── models.py        #   Room, Player, Team, scoring rules
│   └── registry.py      #   in-memory room + sid index
├── services/            # application logic
│   ├── ai_client.py     #   GigaChat + fallback bank + parsing
│   ├── accounts.py      #   users, XP, coins
│   ├── leaderboard.py   #   scores, bans, history
│   ├── ugc.py           #   user questions + moderation
│   ├── campaign.py      #   levels and progression
│   ├── achievements.py  #   achievement catalogue
│   └── learn.py         #   URL/text → questions
├── realtime/            # Socket.IO
│   ├── socket_events.py #   all client/server events
│   ├── chat.py          #   bounded chat store
│   └── scheduler.py     #   async-mode-aware background tasks
└── http_api/            # REST blueprints
    ├── core.py auth.py rooms.py leaderboard_routes.py ugc_routes.py
    └── campaign_routes.py learn_routes.py profile_routes.py
        cheat_routes.py admin_routes.py helpers.py
templates/index.html     # SPA shell
static/js/game.js        # client (views, socket, rendering)
static/css/style.css     # design system
static/manifest.json     # PWA manifest
static/js/sw.js          # service worker
tests/                   # pytest suite (190 tests)
```

Local setup

```
git clone https://github.com/t-1mka/BrainStorm.git
cd BrainStorm

python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install -r requirements.txt
cp .env.example .env             # then edit .env (see below)

python run.py                    # http://localhost:5000
```

On Windows you can instead double-click **`start.bat`**, which checks Python, creates `.env`, installs dependencies, probes GigaChat and prints the LAN URL for phones.

Environment variables (`.env`)

Variable	Default	Description
`ENV`	`development`	`development` | `production`. Production triggers config validation.
`HOST` / `PORT`	`0.0.0.0` / `5000`	Dev-server bind address.
`DEBUG`	`false`	Flask debug + auto-reload. Must be `false` in production.
`LOG_LEVEL`	`INFO`	`DEBUG` | `INFO` | `WARNING` | `ERROR`.
`SOCKETIO_ASYNC_MODE`	auto	`eventlet` | `threading` | `gevent`. Auto = eventlet if installed, else threading.
`SECRET_KEY`	— (required in prod)	Session signing key. No hardcoded default: blank ⇒ random per-process in dev, **startup error in production**. Generate: `python -c "import secrets;print(secrets.token_urlsafe(48))"`.
`SESSION_COOKIE_SECURE`	`false`	Set `true` when served over HTTPS.
`CORS_ORIGINS`	`*`	Comma-separated allowed Socket.IO origins. Use explicit origins in production.
`TRUST_PROXY`	`false`	Trust `X-Forwarded-For` (only behind a trusted proxy).
`RATE_LIMIT_ENABLED`	`true`	Toggle rate limiting globally.
`AUTH_RATE_LIMIT`	`10`	Auth attempts per IP per window.
`ADMIN_SECRET_KEY`	— (required in prod)	Admin panel key. No hardcoded default.
`CHEAT_TESTER_CODE`	— (required in prod)	Tester/cheat mode code. No hardcoded default.
`GIGACHAT_CLIENT_ID`	—	Optional GigaChat client id.
`GIGACHAT_CREDENTIALS`	—	GigaChat authorization key. Empty ⇒ offline fallback bank.
`GIGACHAT_MODEL` / `GIGACHAT_SCOPE`	`GigaChat` / `GIGACHAT_API_PERS`	GigaChat model and scope.
`DATA_DIR`	`data`	SQLite directory (auto-created).
`MAX_ROOMS`	`2000`	Hard cap on concurrent rooms.
`ROOM_IDLE_TIMEOUT` / `ROOM_EMPTY_TIMEOUT`	`90` / `30`	Seconds before idle/empty rooms are reaped.

> **Security:** `.env` is git-ignored and must never be committed. If a secret was ever committed, rotate it — removing the file is not enough.

Tests & quality

```
pip install -r requirements-dev.txt

pytest                                  # run the suite
pytest --cov=app --cov-report=term-missing   # with coverage
ruff check app tests                    # lint
ruff format app tests                   # format
```

The suite runs on the `threading` async mode against a throwaway `DATA_DIR`, so no external services are needed. CI runs lint + tests on Python 3.10–3.12 and builds the Docker image.

Production

Docker

```
docker compose up --build          # http://localhost:5000
```

Render — a blueprint is provided in `render.yaml`; or set manually:

- Build: `pip install -r requirements.txt`
- Start: `gunicorn -c gunicorn.conf.py wsgi:app`
- Health check path: `/health`

> A **single eventlet worker** is intentional. Live rooms live in process memory and each Socket.IO session is bound to the worker that accepted it. Scaling to multiple workers or instances requires moving room state to Redis and enabling sticky sessions — see ROADMAP.md.

API & event reference

- **REST**: `/api/auth/*`, `/api/ugc/*`, `/api/leaderboard`, `/api/campaign/*`, `/api/learn/*`, `/api/profile`, `/api/rooms`, `/api/public_rooms`, `/api/check_session`, `/api/shop/*`, `/api/cheat/*`, `/api/admin/*`; probes `/health`, `/readyz`.
- **Socket.IO client→server**: `create_room`, `join_room`, `rejoin_room`, `leave_room`, `update_settings`, `init_teams`, `draft_pick`, `start_game`, `restart_room`, `submit_answer`, `use_joker`, `get_hint`, `rephrase_question`, `reaction`, `heartbeat`, `set_presentation_mode`, `chat_message`, `chat_clear`, `chat_delete_message`, `cheat_*`, `admin_*`, `svoyaigra_*`.
- The full server→client event list is documented at the top of `app/realtime/socket_events.py`.

License

MIT — see LICENSE. © 2026 geff1778.

🤝 Contributing

Contributions are welcome! Please read CONTRIBUTING.md and the Code of Conduct before opening a pull request. For security issues use SECURITY.md — not a public issue. Planned work lives in ROADMAP.md, and notable changes are tracked in CHANGELOG.md.

🇷🇺 Русский

Что такое BrainStorm?

Мозговой Штурм — самостоятельный сервер для викторин в реальном времени. Ведущий создаёт комнату, делится шестизначным кодом, и все отвечают на одни и те же вопросы одновременно со своих телефонов или ноутбуков. Вопросы генерирует **GigaChat** (Сбербанк) по любой введённой теме, а встроенный офлайн-банк вопросов гарантирует работу игры даже без ключа ИИ.

Возможности

Раздел	Кратко
Игра	Шесть режимов — Классика, FFA, Командный драфт, «На вылет», Кооп, «Своя игра». Адаптивная сложность, стрики, бонус за время, бонусные вопросы, джокеры, подсказки, перефразировка.
ИИ	Генерация вопросов, подсказки и перефразировка через GigaChat; разбор Markdown-JSON; авто-fallback; лимиты на пользователя.
Аккаунты	Регистрация/вход, XP и уровни, монеты, магазин косметики, история матчей, достижения.
Соц.	Чат в комнате с модерацией, реакции, глобальный лидерборд, история комнат.
UGC	Игроки предлагают свои вопросы; админ модерирует очередь.
Кампания	12 одиночных уровней в 3 мирах со звёздами, боссами и наградами.
Эксплуатация	Админ-панель (кик/бан/имперсонация/статистика), чит-режим для тестов, health-проверки, структурные логи, PWA (устанавливается + офлайн-оболочка).

Архитектура

Слои разделены по принципу Clean Architecture:

```
Браузер ──► Flask (фабрика приложения)
               ├─ HTTP REST      app/http_api/*      (10 блюпринтов)
               ├─ Socket.IO      app/realtime/*      (события, чат, планировщик)
               ├─ Сервисы        app/services/*      (ИИ, аккаунты, лидерборд, UGC, кампания)
               └─ Домен          app/domain/*        (Room, Player, Team, очки — без I/O)
                                        │
                                        ▼
                         Хранение  app/db.py
            leaderboard.db                  user_data.db
            (очки, баны, история)           (аккаунты, UGC, кампания, достижения)
```

Правило слоёв: `domain` не делает I/O; `services` зависят от `domain` и `db`; `http_api` и `realtime` — тонкие адаптеры над `services`. `config`, `security`, `errors`, `validation`, `logging_setup` — сквозные модули.

Локальный запуск

```
git clone https://github.com/t-1mka/BrainStorm.git
cd BrainStorm

python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate

pip install -r requirements.txt
cp .env.example .env             # затем отредактируйте .env

python run.py                    # http://localhost:5000
```

В Windows можно запустить **`start.bat`**: он проверит Python, создаст `.env`, установит зависимости, проверит GigaChat и покажет адрес в локальной сети для телефонов.

Переменные окружения

Полная таблица переменных приведена выше в английском разделе. Ключевые: `SECRET_KEY`, `ADMIN_SECRET_KEY`, `CHEAT_TESTER_CODE` (в продакшене обязательны, дефолтов в коде нет), `GIGACHAT_CREDENTIALS` (пусто ⇒ офлайн-банк вопросов), `DATA_DIR`, `ENV`.

> **Безопасность:** `.env` в `.gitignore` и не должен попадать в репозиторий. Если секрет уже был закоммичен — смените его: удаления файла недостаточно.

Тесты и качество

```
pip install -r requirements-dev.txt

pytest                                        # тесты
pytest --cov=app --cov-report=term-missing    # покрытие
ruff check app tests                          # линтер
ruff format app tests                         # форматирование
```

Тесты используют режим `threading` и временный `DATA_DIR`, поэтому внешние сервисы не нужны.

Продакшен

```
docker compose up --build          # http://localhost:5000
```

Для Render используйте `render.yaml` или настройте вручную:

- Build: `pip install -r requirements.txt`
- Start: `gunicorn -c gunicorn.conf.py wsgi:app`
- Health check: `/health`

> **Один eventlet-воркер — это осознанно.** Комнаты живут в памяти процесса, а сессия Socket.IO привязана к принявшему её воркеру. Масштабирование требует переноса состояния комнат в Redis и sticky-сессий — см. ROADMAP.md.

Лицензия

MIT — см. LICENSE. © 2026 geff1778.

Участие в разработке

Мы рады вкладу! Перед созданием pull request прочитайте CONTRIBUTING.md и Кодекс поведения. Об уязвимостях сообщайте по SECURITY.md, а не через публичный issue. Планы развития — в ROADMAP.md, заметные изменения — в CHANGELOG.md.

Команды разработки

Команда	Что делает
`make doctor`	проверка инструментов, версий, зависимостей и того, что `.env` не в git
`make install-dev`	закреплённые зависимости + pytest/ruff
`make test`	190 тестов на изолированном `DATA_DIR`
`make lint`	ruff
`make compose-up` / `make compose-down`	контейнер (данные в томе сохраняются)
`make smoke-test`	`/health` и главная страница
`make docs-check`	полнота документации и спецификаций модулей
`make verify`	lint + тесты + документация

Документация: точка входа — docs/index.md; сборка — BUILD.md; матрица проверок — docs/verification/matrix.md.

> **Важно.** Ранее в репозитории лежал `.env` с рабочим ключом GigaChat. Файл убран из индекса, добавлено сканирование секретов в CI. **Ключи нужно перевыпустить** — см. docs/security/secrets.md.

Управление проектом: `forge.sh` и `forge.bat`

Один скрипт на Unix-подобные системы (`forge.sh`) и один на Windows (`forge.bat`). Интерфейс одинаковый: те же команды, та же логика, одна реализация на каждой платформе — без bash-скриптов внутри Windows.

```
./forge.sh              # интерактивное меню
./forge.sh doctor       # проверить окружение, ничего не меняя
./forge.sh install      # поставить зависимости
./forge.sh start        # запустить в фоне, дождаться готовности, открыть браузер
./forge.sh status       # что запущено и отвечает ли порт
./forge.sh logs api -f  # логи в реальном времени
./forge.sh test         # тесты
./forge.sh stop         # остановить
```

```
forge.bat
forge.bat start
forge.bat status
```

Что умеет

Команда	Действие
`status`	состояние по портам и HTTP, а не «на глаз»
`start [--fg]`	фон (PID-файлы в `.run/`) или передний план
`stop`	останавливает дерево процессов целиком, порт освобождается
`restart`, `logs [имя] [-f]`, `open`	обычные операции
`doctor`	инструменты, версии, `.env`, `.env` в git, занятость порта
`install`, `test`, `lint`, `build`, `verify`	через `make`, если он есть

| `db migrate\|backup\|restore` | база: миграции, копия, восстановление | | `docker up\|down\|logs\|reset` | контейнеры (`down` данные не удаляет) | | `clean [--hard]` | артефакты и логи; `--hard` ещё `.venv` и `node_modules` |

Правила, заложенные в скрипт

- **Ничего не меняет без команды.** `doctor` и `status` только читают.
- **Данные не удаляются.** `stop` и `docker down` сохраняют базу; удаление — только `--hard` или явное подтверждение в `docker reset`.
- **Предполётная проверка.** Если нет `go`/python или занят порт — скрипт говорит об этом сразу, а не через две минуты таймаута.
- **Не висит после падения.** Если процесс умер, запуск прекращается и показывает последние строки лога.
- **Venv главнее системы.** Если есть `.venv`, им пользуются `make`, `pytest` и uvicorn — чтобы `install` и `test` не расходились.

Текущий способ запуска без скрипта: `python run.py` — адрес `http://localhost:5000/health`.

Конфигурация: `.env`

Скрипт читает `.env` **собственным парсером**, а не через `source`: файл приходит из репозитория, а выполнять из него произвольный код нельзя. Поддержаны комментарии `#`, пустые строки, префикс `export`, одинарные и двойные кавычки, CRLF, пробелы вокруг `=` и комментарий после значения.

Приоритет значений: **переменная окружения → `.env` → встроенное значение**. Поэтому `PORT=9000 ./forge.sh start` работает независимо от файла.

Команда	Действие
`env` / `env show`	переменные, источник каждого значения и замаскированные секреты
`env init [--force]`	создать `.env` из `.env.example`: заглушки секретов заменяются случайными, права `600`, прежний файл — в `.env.bak`
`env set KEY=VALUE`	записать или заменить одну переменную (атомарно, через временный файл)
`env check`	сверка с `.env.example`: пропавшие переменные, пустые и «примерные» секреты, `.env` в `.gitignore`
`env edit`	открыть файл в `$EDITOR` (fallback: nano → vim → vi)
`env path`, `env reload`	путь к файлу и перечитывание без перезапуска

Секреты в выводе маскируются: `ADMIN_CODE=ch…de (31 симв.)`, пароль внутри строки подключения — `postgres://user:***@host`. Имена вида `*_MINUTES`, `*_URL`, `*_PORT` секретами не считаются: маскировать число минут бессмысленно.

Путь к файлу переопределяется: `FORGE_ENV_FILE=/etc/app.env ./forge.sh status`. В Windows-версии те же команды: `forge.bat env init`, `forge.bat env check`. Ограничение `cmd.exe`: значения, содержащие `%` и `!`, не поддерживаются.

Флаги и автодополнение

```
./forge.sh -h | --help            # справка
./forge.sh -V | --version         # версия скрипта, коммит, состояние .env
./forge.sh -q status              # молчать об успехах
./forge.sh --no-color status      # без ANSI (то же делает NO_COLOR=1)
./forge.sh --json status          # машинный вывод для скриптов и мониторинга

eval "$(./forge.sh completion bash)"   # автодополнение для bash
./forge.sh completion zsh > ~/.zfunc/_forge   # для zsh
```

`test`, `lint`, `build` и `verify` запускаются в окружении **без** переменных из `.env`: иначе локальные проверки вели бы себя иначе, чем CI, и тесты вида «без ключа должен быть fallback» перестали бы что-либо проверять.

Запуск в этом проекте

```
./forge.sh install && ./forge.sh start
```